Vous : Bonsoir. Pouvez vous créer en utilisant leaflet de Javascript avec jquery une application comme GOOGLE EARTH qui permet de visualiser tout ce qui se trouve sur la planete Terre ? De preference pouriez vous egalement ajouter des boutons de zoom + et de zoom - avec une transmission image par satelite en temps réel ? N'oubliez pas d'afficher les legendes et une option permettant de choisir entre IMAGE PAR SATELITE et IMAGE CROQUIS

Gemini a dit

Bonsoir ! C'est un excellent projet. Créer un visualiseur de cartes interactif avec Leaflet et jQuery est tout à fait réalisable et constitue une très bonne base pour une application de cartographie.

Avant de passer au code, il y a une petite précision technique importante à apporter concernant votre demande : **il n'existe pas d'API publique et gratuite permettant d'obtenir des images satellites de toute la planète en "temps réel"**. Ce que l'on voit dans les films (un zoom en direct depuis l'espace) n'est pas accessible au grand public. Les services comme Google Earth, Bing Maps ou Esri utilisent des images capturées par des satellites ou des avions qui sont ensuite assemblées. Ces images sont mises à jour périodiquement (parfois après quelques mois, voire quelques années selon les zones). Nous allons donc utiliser la couche satellite d'Esri (très détaillée et proche de ce que propose Google Earth) pour l'option "Image par satellite".

De plus, Leaflet est une bibliothèque 2D (comme Google Maps). Si vous souhaitiez un globe 3D qui tourne, il faudrait utiliser une bibliothèque comme **CesiumJS**. Cependant, Leaflet répondra parfaitement à vos autres critères.

Voici comment construire votre application avec les boutons de zoom (inclus par défaut dans Leaflet), le choix des couches et une légende.

1. Le code HTML

Créez un fichier `index.html`. Ce code charge Leaflet, jQuery, et prépare le conteneur de la carte.

HTML
```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Mon Application Type Google Earth</title>

    <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />

    <style>
        body { margin: 0; padding: 0; }
        /* La carte doit prendre tout l'écran */
        #map { width: 100vw; height: 100vh; }

        /* Style pour la légende personnalisée */
```

```
        .info.legend {
            background: white;
            padding: 10px;
            border-radius: 5px;
            box-shadow: 0 0 15px rgba(0,0,0,0.2);
            font-family: Arial, sans-serif;
            font-size: 14px;
            line-height: 1.5;
        }
        .legend-color {
            display: inline-block;
            width: 15px;
            height: 15px;
            margin-right: 8px;
            border: 1px solid #000;
        }
    </style>
</head>
<body>

    <div id="map"></div>

    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>
    <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>
    <script src="app.js"></script>
</body>
</html>
```

2. Le code JavaScript (`app.js`)

Créez un fichier `app.js` dans le même dossier. Nous allons utiliser jQuery pour initialiser la carte une fois la page chargée.

JavaScript
```
$(document).ready(function() {
    // 1. Initialisation de la carte
    // Coordonnées de départ (latitude, longitude) et niveau de zoom
    var map = L.map('map').setView([46.603354, 1.888334], 5); // Centré sur la France par défaut

    // Les boutons Zoom + et Zoom - sont ajoutés automatiquement en haut à gauche par Leaflet.

    // 2. Définition des couches de fond (Tuiles)

    // Option A : IMAGE CROQUIS (OpenStreetMap)
    var coucheCroquis = L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
```

```
        maxZoom: 19,
        attribution: '© OpenStreetMap contributors'
    });

    // Option B : IMAGE PAR SATELITE (Esri World Imagery)
    var coucheSatellite =
L.tileLayer('https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}', {
        maxZoom: 19,
        attribution: 'Tiles © Esri — Source: Esri, i-cubed, USDA, USGS, AEX, GeoEye, Getmapping, Aerogrid, IGN, IGP, UPR-EGP,
and the GIS User Community'
    });

    // 3. Application de la couche par défaut
    coucheSatellite.addTo(map);

    // 4. Création du contrôleur pour choisir entre les deux images
    var baseMaps = {
        "IMAGE PAR SATELITE": coucheSatellite,
        "IMAGE CROQUIS": coucheCroquis
    };

    L.control.layers(baseMaps).addTo(map);

    // 5. Ajout d'une légende en bas à droite
    var legend = L.control({position: 'bottomright'});

    legend.onAdd = function (map) {
        // Création d'un div HTML pour la légende
        var div = L.DomUtil.create('div', 'info legend');

        // Contenu HTML de la légende
        div.innerHTML += '<h4>Légende</h4>';
        div.innerHTML += '<i class="legend-color" style="background: #a3ccff"></i> Eau<br>';
        div.innerHTML += '<i class="legend-color" style="background: #c2e699"></i> Végétation<br>';
        div.innerHTML += '<i class="legend-color" style="background: #e0e0e0"></i> Zones Urbaines<br>';

        return div;
    };

    legend.addTo(map);
});
```

Ce que fait ce code :

- **L.map('map')** : Crée la carte interactive. Les boutons de zoom (le **+** et le **-**) sont générés nativement.

- **Esri World Imagery** : Fournit une vue satellite mondiale de très haute qualité.
- **OpenStreetMap** : Fournit une vue "croquis" ou carte standard.
- **L.control.layers** : Ajoute l'icône en haut à droite permettant à l'utilisateur de basculer instantanément entre "IMAGE PAR SATELITE" et "IMAGE CROQUIS".
- **La Légende** : Un encart personnalisé est injecté en bas à droite pour décrire les couleurs de base (vous pourrez modifier ces couleurs et textes selon le contexte exact de votre projet).

  Souhaitez-vous que nous ajoutions une fonctionnalité de recherche (une barre de saisie) avec l'API Nominatim pour permettre à l'utilisateur de taper le nom d'une ville et de s'y téléporter directement ?

  Vous avez dit

  oui

Gemini a dit

Super ! L'ajout d'une barre de recherche va rendre l'application beaucoup plus dynamique. Pour cela, nous allons utiliser **Nominatim**, le moteur de recherche d'OpenStreetMap (un processus que l'on appelle le *géocodage* : transformer un nom en coordonnées GPS).

Voici les modifications à apporter à vos trois blocs de code pour intégrer cette fonctionnalité.

1. Mise à jour du HTML

Ajoutez simplement le bloc de la barre de recherche juste au-dessus du conteneur de votre carte (`<div id="map"></div>`).

```HTML
    <div id="search-box">
        <input type="text" id="search-input" placeholder="Rechercher une ville, un pays...">
        <button id="search-button">Aller</button>
    </div>

    <div id="map"></div>
```

2. Mise à jour du CSS

Ajoutez ces styles dans votre balise `<style>` pour positionner la barre de recherche par-dessus la carte, sans bloquer les boutons de zoom.

```CSS
        /* Style de la barre de recherche */
        #search-box {
```

```
        position: absolute;
        top: 15px;
        left: 60px; /* Décalé pour ne pas cacher le zoom */
        z-index: 1000; /* Permet d'être au-dessus de la carte Leaflet */
        background: white;
        padding: 8px;
        border-radius: 5px;
        box-shadow: 0 2px 10px rgba(0,0,0,0.3);
        display: flex;
        gap: 5px;
    }

    #search-input {
        padding: 6px;
        width: 250px;
        border: 1px solid #ccc;
        border-radius: 3px;
    }

    #search-button {
        padding: 6px 12px;
        cursor: pointer;
        background-color: #007bff;
        color: white;
        border: none;
        border-radius: 3px;
    }

    #search-button:hover {
        background-color: #0056b3;
    }
```

3. Mise à jour du JavaScript (`app.js`)

Ajoutez ce code à la fin de votre fichier `app.js` (mais toujours à l'intérieur de la fonction `$(document).ready(function() { ... });`).

Ce script utilise **jQuery** pour intercepter le clic sur le bouton, envoyer une requête à l'API Nominatim, et utiliser la fonction `flyTo` de Leaflet pour créer une animation de vol vers la destination.

JavaScript
```
    // --- NOUVEAU CODE : FONCTIONNALITÉ DE RECHERCHE ---

    var searchMarker; // Variable pour garder la trace du marqueur actuel
```

```
function lancerRecherche() {
    var query = $('#search-input').val();

    if (query.trim() !== '') {
        // Requête AJAX vers l'API gratuite d'OpenStreetMap (Nominatim)
        $.ajax({
            url: 'https://nominatim.openstreetmap.org/search',
            type: 'GET',
            data: {
                format: 'json',
                q: query,
                limit: 1 // On prend uniquement le résultat le plus pertinent
            },
            success: function(data) {
                if (data && data.length > 0) {
                    var result = data[0];
                    var lat = result.lat;
                    var lon = result.lon;

                    // Animation de "vol" vers les nouvelles coordonnées
                    map.flyTo([lat, lon], 13, {
                        duration: 2 // Durée de l'animation en secondes
                    });

                    // Si un marqueur existe déjà, on l'enlève
                    if (searchMarker) {
                        map.removeLayer(searchMarker);
                    }

                    // Ajout d'un nouveau marqueur avec le nom exact trouvé
                    searchMarker = L.marker([lat, lon]).addTo(map)
                        .bindPopup('<b>Résultat :</b><br>' + result.display_name)
                        .openPopup();
                } else {
                    alert("Lieu introuvable. Essayez avec un autre nom.");
                }
            },
            error: function() {
                alert("Erreur de connexion au service de recherche.");
            }
        });
    }
}

// Déclencher la recherche au clic sur le bouton
$('#search-button').click(function() {
```

```
        lancerRecherche();
    });

    // Déclencher la recherche en appuyant sur la touche "Entrée"
    $('#search-input').keypress(function(e) {
        if (e.which == 13) {
            lancerRecherche();
        }
    });
```

Désormais, si vous ouvrez votre fichier HTML, vous pouvez taper "Paris", "Mont Fuji", ou n'importe quelle adresse, et la carte fera un zoom fluide jusqu'à la destination avec une petite étiquette descriptive !

Souhaitez-vous que nous transformions cette application web en un véritable logiciel de bureau exécutable (`.exe`) en intégrant ce code HTML/JS à l'intérieur d'une interface graphique conçue avec Python et PyQt6 (via `QWebEngineView`) ?

Vous avez dit

Pouvez vous donner les codes complets et corrigés de tous les fichiers ? svp

Gemini a dit

Voici les codes complets et structurés. Pour garder votre projet propre et facile à maintenir, j'ai séparé le code en trois fichiers distincts : un pour la structure (HTML), un pour le design (CSS) et un pour la logique (JavaScript).

Placez ces trois fichiers dans le même dossier et ouvrez simplement `index.html` dans votre navigateur.

1. Le fichier `index.html`

Ce fichier charge les bibliothèques nécessaires (Leaflet, jQuery), relie vos fichiers CSS et JavaScript, et prépare l'interface de base.

```HTML
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Application Cartographique</title>

    <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />

    <link rel="stylesheet" href="style.css" />
</head>
<body>
```

```
    <div id="search-box">
        <input type="text" id="search-input" placeholder="Rechercher une ville, un pays...">
        <button id="search-button">Aller</button>
    </div>

    <div id="map"></div>

    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

    <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>

    <script src="app.js"></script>
</body>
</html>
```

---

2. Le fichier `style.css`

Ce fichier gère l'apparence visuelle. Il s'assure que la carte prend tout l'écran, que la barre de recherche est positionnée correctement par-dessus la carte, et stylise la légende.

```
CSS
/* Réinitialisation des marges */
body {
    margin: 0;
    padding: 0;
}

/* La carte prend tout l'écran */
#map {
    width: 100vw;
    height: 100vh;
}

/* Style de la barre de recherche */
#search-box {
    position: absolute;
    top: 15px;
    left: 60px; /* Décalé pour ne pas cacher les boutons de zoom natifs */
    z-index: 1000; /* Permet d'être au-dessus de la carte Leaflet */
    background: white;
    padding: 8px;
```

```
    border-radius: 5px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.3);
    display: flex;
    gap: 5px;
}

#search-input {
    padding: 6px;
    width: 250px;
    border: 1px solid #ccc;
    border-radius: 3px;
}

#search-button {
    padding: 6px 12px;
    cursor: pointer;
    background-color: #007bff;
    color: white;
    border: none;
    border-radius: 3px;
}

#search-button:hover {
    background-color: #0056b3;
}

/* Style pour la légende personnalisée */
.info.legend {
    background: white;
    padding: 10px;
    border-radius: 5px;
    box-shadow: 0 0 15px rgba(0,0,0,0.2);
    font-family: Arial, sans-serif;
    font-size: 14px;
    line-height: 1.5;
}

.legend-color {
    display: inline-block;
    width: 15px;
    height: 15px;
    margin-right: 8px;
    border: 1px solid #000;
}
```

3. Le fichier `app.js`

Ce fichier contient toute la logique : l'affichage des tuiles (satellite et croquis), le menu de sélection, la légende et les requêtes AJAX vers Nominatim pour la recherche de lieux.

JavaScript
```
$(document).ready(function() {
    // 1. Initialisation de la carte (Centrée sur la France par défaut)
    var map = L.map('map').setView([46.603354, 1.888334], 5);

    // 2. Définition des couches de fond
    var coucheCroquis = L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
        maxZoom: 19,
        attribution: '© OpenStreetMap contributors'
    });

    var coucheSatellite =
L.tileLayer('https://server.arcgisonline.com/ArcGIS/rest/services/World_Imagery/MapServer/tile/{z}/{y}/{x}', {
        maxZoom: 19,
        attribution: 'Tiles © Esri'
    });

    // 3. Application de la couche satellite par défaut
    coucheSatellite.addTo(map);

    // 4. Création du contrôleur pour choisir entre les deux images
    var baseMaps = {
        "IMAGE PAR SATELITE": coucheSatellite,
        "IMAGE CROQUIS": coucheCroquis
    };
    L.control.layers(baseMaps).addTo(map);

    // 5. Ajout d'une légende en bas à droite
    var legend = L.control({position: 'bottomright'});

    legend.onAdd = function (map) {
        var div = L.DomUtil.create('div', 'info legend');
        div.innerHTML += '<h4>Légende</h4>';
        div.innerHTML += '<i class="legend-color" style="background: #a3ccff"></i> Eau<br>';
        div.innerHTML += '<i class="legend-color" style="background: #c2e699"></i> Végétation<br>';
        div.innerHTML += '<i class="legend-color" style="background: #e0e0e0"></i> Zones Urbaines<br>';
        return div;
    };
    legend.addTo(map);
```

```
// 6. Fonctionnalité de recherche
var searchMarker; // Garde la trace du marqueur actuel

function lancerRecherche() {
    var query = $('#search-input').val();

    if (query.trim() !== '') {
        // Requête AJAX vers Nominatim
        $.ajax({
            url: 'https://nominatim.openstreetmap.org/search',
            type: 'GET',
            data: {
                format: 'json',
                q: query,
                limit: 1 // On prend le résultat le plus pertinent
            },
            success: function(data) {
                if (data && data.length > 0) {
                    var result = data[0];
                    var lat = result.lat;
                    var lon = result.lon;

                    // Animation vers la destination
                    map.flyTo([lat, lon], 13, {
                        duration: 2
                    });

                    // Supprime l'ancien marqueur s'il existe
                    if (searchMarker) {
                        map.removeLayer(searchMarker);
                    }

                    // Ajoute le nouveau marqueur
                    searchMarker = L.marker([lat, lon]).addTo(map)
                        .bindPopup('<b>Résultat :</b><br>' + result.display_name)
                        .openPopup();
                } else {
                    alert("Lieu introuvable. Essayez avec un autre nom.");
                }
            },
            error: function() {
                alert("Erreur de connexion au service de recherche.");
            }
        });
    }
}
```

```
    // Lancer la recherche au clic
    $('#search-button').click(function() {
        lancerRecherche();
    });

    // Lancer la recherche avec la touche "Entrée"
    $('#search-input').keypress(function(e) {
        if (e.which == 13) {
            lancerRecherche();
        }
    });
});
```

Une fois testée dans votre navigateur, cette interface web peut facilement être embarquée dans une application de bureau lourde. Voulez-vous que je vous fournisse le script Python utilisant `QWebEngineView` de PyQt6 pour encapsuler ces fichiers HTML/JS et en faire un logiciel de bureau indépendant ?

Vous avez dit

Question : Donc il n'est pas possible d'obtenir des images qui sont à jour ?

Gemini a dit

C'est une excellente question et une confusion très fréquente. Il faut faire une distinction importante entre **"en temps réel"** (comme une vidéo en direct) et **"récent"** (mis à jour il y a quelques jours ou quelques semaines).

Il est tout à fait possible d'obtenir des images satellites très récentes, mais il n'est pas possible d'avoir un flux "en direct" couvrant toute la planète en libre accès.

Voici comment fonctionne la réalité des images satellites et comment vous pouvez obtenir les données les plus à jour.

1. Pourquoi le "Temps Réel" global n'existe pas

Dans les films, on voit souvent une caméra faire un zoom continu depuis l'espace jusqu'à la plaque d'immatriculation d'une voiture en temps réel. Dans la réalité, plusieurs obstacles techniques rendent cela impossible pour une carte globale :

- **Le cycle des orbites :** Les satellites qui prennent des photos en haute résolution ne sont pas fixes au-dessus d'un point (géostationnaires) ; ils tournent autour de la Terre. Ils ne repassent au-dessus du même endroit que tous les quelques jours.
- **Le traitement des données :** Une image satellite brute ne ressemble pas à ce qu'on voit sur Google Earth. Elle doit être téléchargée sur Terre, corrigée (pour enlever la distorsion de l'atmosphère et l'angle de vue), assemblée avec d'autres images, et compressée. C'est un traitement lourd.

- **La couverture nuageuse :** À tout moment, environ 67% de la Terre est recouverte de nuages. Les fournisseurs d'images (comme Esri ou Google) attendent d'avoir une image claire et sans nuage d'une zone avant de mettre à jour leur carte principale.

2. Comment obtenir des images "Récentes" (Mises à jour)

Si vous avez besoin d'images datant de quelques jours, la couche Esri utilisée précédemment ne suffira pas (elle est mise à jour tous les 1 à 3 ans pour avoir une carte mondiale sans nuages parfaite). Il faut alors se tourner vers des données brutes, souvent issues de programmes spatiaux :

- **Le programme Copernicus (Sentinel-2) :** C'est une initiative européenne publique et gratuite. Les satellites Sentinel-2 photographient la Terre entière **tous les 5 jours**. Les images ne sont pas aussi précises que celles de Google Earth (la résolution est de 10 mètres par pixel, on voit les bâtiments, mais pas les voitures), mais elles sont extrêmement à jour.
- **Landsat (NASA/USGS) :** Un système américain similaire, avec des mises à jour régulières, très utile pour observer les changements environnementaux, l'agriculture ou l'urbanisme.
- **Les services commerciaux (Planet, Maxar) :** Des entreprises privées possèdent des constellations de centaines de mini-satellites capables de photographier n'importe quel point du globe **tous les jours** en haute résolution. Cependant, l'accès à leurs API coûte des milliers d'euros.

3. L'approche technique pour votre projet

Pour intégrer des images très récentes dans un logiciel, l'approche standard consiste à interroger directement les API scientifiques. Il est tout à fait possible d'écrire un script Python qui va requêter l'API du programme Copernicus (via des bibliothèques comme `sentinelsat`), télécharger l'image d'une zone géographique précise datant de la semaine dernière, et la superposer sur votre carte Leaflet.

Souhaitez-vous que nous explorions l'intégration de l'API de Sentinel-2 pour récupérer et afficher ces images récentes, ou préférez-vous d'abord finaliser l'interface de notre application cartographique actuelle en la transformant en exécutable de bureau avec PyQt6 ?

Vous avez dit

Il est tout à fait possible d'écrire un script Python qui va requêter l'API du programme Copernicus (via des bibliothèques comme sentinelsat), télécharger l'image d'une zone géographique précise datant de la semaine dernière, et la superposer sur votre carte Leaflet.

Gemini a dit

C'est une excellente direction ! C'est exactement l'approche utilisée par les professionnels de la cartographie pour obtenir des données à jour.

Avant de vous donner le code, je dois faire une petite précision technique pour vous éviter des heures de frustration. Bien que des bibliothèques Python comme `sentinelsat` existent, elles sont conçues pour télécharger les données "brutes". Une seule image brute de Sentinel-2 pèse

souvent plus d'**1 Go** et nécessite d'être traitée (combinaison de bandes de couleurs, géoréférencement) avant de pouvoir l'afficher. Ce n'est pas idéal pour une application fluide.

La méthode la plus performante et moderne consiste à utiliser un service **WMS (Web Map Service)**. Au lieu de télécharger des gigaoctets de données, l'API de Copernicus génère directement les "tuiles" d'images récentes et les envoie à Leaflet.

Pour encapsuler tout cela dans un logiciel de bureau robuste et autonome, nous allons utiliser **Python avec la bibliothèque PyQt6** et son module web.

Voici comment mettre en place cette solution complète.

1. Mise à jour du fichier `app.js` (La couche Sentinel-2)

Nous allons ajouter une nouvelle couche WMS provenant de Sentinel Hub (l'interface d'accès aux données Copernicus).

Note : Pour que les images s'affichent réellement, vous devrez créer un compte gratuit sur le site de Sentinel Hub pour obtenir un `INSTANCE_ID`. Dans le code ci-dessous, j'utilise une URL de démonstration.

Modifiez la partie de définition des couches dans votre `app.js` :

JavaScript
```javascript
    // ... (Code précédent d'initialisation de la carte) ...

    var coucheCroquis = L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
        maxZoom: 19,
        attribution: '© OpenStreetMap contributors'
    });

    // Nouvelle couche : Images récentes de Sentinel-2 via WMS
    // Remplacez "VOTRE_INSTANCE_ID" par l'ID obtenu sur Sentinel Hub
    var urlSentinel = 'https://services.sentinel-hub.com/ogc/wms/VOTRE_INSTANCE_ID';

    var coucheSentinelRecent = L.tileLayer.wms(urlSentinel, {
        layers: 'TRUE-COLOR-S2L2A', // Demande l'image en couleurs réelles
        format: 'image/jpeg',
        transparent: false,
        maxZoom: 16, // Les satellites ont une limite de zoom physique (10m/pixel)
        attribution: '© Copernicus Sentinel Data'
    });

    // Application de la couche Sentinel par défaut
    coucheSentinelRecent.addTo(map);

    // Mise à jour du contrôleur
```

```
    var baseMaps = {
        "IMAGE RECENTE (Copernicus)": coucheSentinelRecent,
        "IMAGE CROQUIS": coucheCroquis
    };
    L.control.layers(baseMaps).addTo(map);

    // ... (Suite du code pour la légende et la recherche) ...
```

2. Le script Python (L'application de bureau PyQt6)

Ce script va créer une fenêtre native Windows/Mac/Linux et charger votre interface Leaflet à l'intérieur.

Vous aurez besoin d'installer PyQt6 et son composant web. Ouvrez votre terminal et tapez : `pip install PyQt6 PyQt6-WebEngine`

Ensuite, créez un fichier `main.py` dans le même dossier que vos fichiers HTML/JS/CSS :

```python
import sys
import os
from PyQt6.QtWidgets import QApplication, QMainWindow, QVBoxLayout, QWidget
from PyQt6.QtWebEngineWidgets import QWebEngineView
from PyQt6.QtCore import QUrl

class CartographieApp(QMainWindow):
    def __init__(self):
        super().__init__()

        # Configuration de la fenêtre principale
        self.setWindowTitle("Logiciel de Cartographie - Données Satellites Récentes")
        self.setGeometry(100, 100, 1200, 800) # x, y, largeur, hauteur

        # Création du composant navigateur web
        self.navigateur = QWebEngineView()

        # Récupération du chemin absolu vers le fichier index.html
        chemin_dossier_actuel = os.path.dirname(os.path.abspath(__file__))
        chemin_html = os.path.join(chemin_dossier_actuel, "index.html")

        # Chargement de la page HTML locale dans le navigateur
        self.navigateur.setUrl(QUrl.fromLocalFile(chemin_html))

        # Définition du navigateur comme widget central de la fenêtre
        self.setCentralWidget(self.navigateur)

if __name__ == "__main__":
```

```
    # Initialisation de l'application Qt
    app = QApplication(sys.argv)

    # Création et affichage de la fenêtre
    fenetre = CartographieApp()
    fenetre.show()

    # Boucle d'exécution principale
    sys.exit(app.exec())
```

Résultat

1. Assurez-vous que `main.py`, `index.html`, `style.css` et `app.js` sont tous dans le même dossier.
2. Lancez le script Python (`python main.py`).
3. Une fenêtre de logiciel de bureau va s'ouvrir, affichant votre carte interactive. Les requêtes Javascript iront interroger l'API Copernicus pour afficher les dalles d'images satellitaires les plus récentes.

   Souhaitez-vous que je vous guide sur la procédure exacte (qui est gratuite) pour obtenir votre "INSTANCE_ID" sur le portail de Sentinel Hub afin que les images s'affichent correctement ?

   Vous avez dit

   Oui je suis bien d'accord ! Pouvez vous egalement donner tous les fichiers corrigés et complets ? svp

Gemini a dit

Parfait ! Allons-y étape par étape. Je vais d'abord vous expliquer comment obtenir votre accès gratuit aux images satellites, puis je vous fournirai l'intégralité des fichiers prêts à l'emploi.

Étape A : Obtenir votre "INSTANCE_ID" Sentinel Hub (Gratuit)

Pour que la couche d'images satellites récentes s'affiche, vous devez dire aux serveurs de Copernicus qui vous êtes.

1. Allez sur le site **Sentinel Hub** et créez un compte gratuit (cliquez sur "Sign Up").
2. Une fois connecté, allez dans le tableau de bord (Dashboard) et cherchez l'onglet **"Configuration Utility"** (Utilitaire de configuration).
3. Cliquez sur le bouton pour créer une nouvelle configuration (New Configuration).
4. Donnez-lui un nom (par exemple : "MonAppCarto") et choisissez comme base **"Sentinel-2 L2A"** (ce sont les images avec les couleurs naturelles corrigées pour l'atmosphère).

5. Sauvegardez. Vous verrez alors apparaître sur votre configuration un identifiant alphanumérique appelé **"Instance ID"** (une suite de lettres et de chiffres).
6. Copiez cet identifiant. C'est lui que vous collerez dans le fichier `app.js` à la place de `VOTRE_INSTANCE_ID`.

Étape B : Les fichiers complets de votre projet

Voici les 4 fichiers nécessaires pour faire fonctionner votre logiciel. Placez-les tous dans le même dossier sur votre ordinateur.

1. Le fichier `index.html`

Ce fichier reste la structure de base qui relie vos différents éléments.

HTML
```html
<!DOCTYPE html>
<html lang="fr">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Application Cartographique Avancée</title>

    <link rel="stylesheet" href="https://unpkg.com/leaflet@1.9.4/dist/leaflet.css" />

    <link rel="stylesheet" href="style.css" />
</head>
<body>

    <div id="search-box">
        <input type="text" id="search-input" placeholder="Rechercher une ville, un pays...">
        <button id="search-button">Aller</button>
    </div>

    <div id="map"></div>

    <script src="https://code.jquery.com/jquery-3.7.1.min.js"></script>

    <script src="https://unpkg.com/leaflet@1.9.4/dist/leaflet.js"></script>

    <script src="app.js"></script>
</body>
</html>
```

2. Le fichier `style.css`

Il gère l'esthétique de l'application, en s'assurant que la carte occupe tout l'espace de la fenêtre de votre logiciel.

```CSS
/* Réinitialisation de base */
body, html {
    margin: 0;
    padding: 0;
    width: 100%;
    height: 100%;
    overflow: hidden; /* Empêche l'apparition de barres de défilement inutiles */
}

/* La carte prend tout l'écran */
#map {
    width: 100vw;
    height: 100vh;
}

/* Style de la barre de recherche */
#search-box {
    position: absolute;
    top: 15px;
    left: 60px;
    z-index: 1000;
    background: white;
    padding: 8px;
    border-radius: 5px;
    box-shadow: 0 2px 10px rgba(0,0,0,0.3);
    display: flex;
    gap: 5px;
}

#search-input {
    padding: 6px;
    width: 250px;
    border: 1px solid #ccc;
    border-radius: 3px;
    font-family: Arial, sans-serif;
}

#search-button {
    padding: 6px 12px;
    cursor: pointer;
```

```
    background-color: #007bff;
    color: white;
    border: none;
    border-radius: 3px;
    font-family: Arial, sans-serif;
}

#search-button:hover {
    background-color: #0056b3;
}

/* Style de la légende */
.info.legend {
    background: white;
    padding: 10px;
    border-radius: 5px;
    box-shadow: 0 0 15px rgba(0,0,0,0.2);
    font-family: Arial, sans-serif;
    font-size: 14px;
    line-height: 1.5;
}

.legend-color {
    display: inline-block;
    width: 15px;
    height: 15px;
    margin-right: 8px;
    border: 1px solid #000;
}
```

3. Le fichier `app.js`

N'oubliez pas de remplacer `VOTRE_INSTANCE_ID` à la ligne 11 par l'identifiant récupéré à l'Étape A.

JavaScript

```
$(document).ready(function() {
    // 1. Initialisation de la carte (Centrée sur la France)
    var map = L.map('map').setView([46.603354, 1.888334], 6);

    // 2. Définition de la couche Croquis (OpenStreetMap)
    var coucheCroquis = L.tileLayer('https://{s}.tile.openstreetmap.org/{z}/{x}/{y}.png', {
        maxZoom: 19,
        attribution: '© OpenStreetMap contributors'
    });
```

```
// 3. Définition de la couche Satellite Récente (Copernicus / Sentinel Hub)
var instanceId = 'VOTRE_INSTANCE_ID'; // <--- REMPLACEZ CECI
var urlSentinel = 'https://services.sentinel-hub.com/ogc/wms/' + instanceId;

var coucheSentinelRecent = L.tileLayer.wms(urlSentinel, {
    layers: 'TRUE-COLOR-S2L2A', // Image en couleurs naturelles
    format: 'image/jpeg',
    transparent: false,
    maxZoom: 16, // Limite de résolution du satellite (10m)
    attribution: '© Copernicus Sentinel Data'
});

// Application de la couche satellite par défaut
coucheSentinelRecent.addTo(map);

// 4. Création du menu de sélection des couches (en haut à droite)
var baseMaps = {
    "IMAGE SATELLITE (Récente)": coucheSentinelRecent,
    "IMAGE CROQUIS": coucheCroquis
};
L.control.layers(baseMaps).addTo(map);

// 5. Ajout de la légende (en bas à droite)
var legend = L.control({position: 'bottomright'});
legend.onAdd = function (map) {
    var div = L.DomUtil.create('div', 'info legend');
    div.innerHTML += '<h4>Légende</h4>';
    div.innerHTML += '<i class="legend-color" style="background: #a3ccff"></i> Eau<br>';
    div.innerHTML += '<i class="legend-color" style="background: #c2e699"></i> Végétation<br>';
    div.innerHTML += '<i class="legend-color" style="background: #e0e0e0"></i> Zones Urbaines<br>';
    return div;
};
legend.addTo(map);

// 6. Fonctionnalité de recherche (Géocodage avec Nominatim)
var searchMarker;

function lancerRecherche() {
    var query = $('#search-input').val();

    if (query.trim() !== '') {
        $.ajax({
            url: 'https://nominatim.openstreetmap.org/search',
            type: 'GET',
            data: {
```

```
                    format: 'json',
                    q: query,
                    limit: 1
                },
                success: function(data) {
                    if (data && data.length > 0) {
                        var result = data[0];
                        var lat = result.lat;
                        var lon = result.lon;

                        // Vol vers la destination
                        map.flyTo([lat, lon], 12, { duration: 2 });

                        // Mise à jour du marqueur
                        if (searchMarker) {
                            map.removeLayer(searchMarker);
                        }
                        searchMarker = L.marker([lat, lon]).addTo(map)
                            .bindPopup('<b>Résultat :</b><br>' + result.display_name)
                            .openPopup();
                    } else {
                        alert("Lieu introuvable. Essayez de préciser votre recherche.");
                    }
                },
                error: function() {
                    alert("Erreur de réseau lors de la recherche.");
                }
            });
        }
    }

    // Déclencheurs de recherche
    $('#search-button').click(function() { lancerRecherche(); });
    $('#search-input').keypress(function(e) {
        if (e.which == 13) { lancerRecherche(); } // Touche Entrée
    });
});
```

4. Le fichier `main.py`

Ce script crée l'interface graphique de votre logiciel avec PyQt6 et y intègre la carte interactive. Assurez-vous d'avoir exécuté `pip install PyQt6 PyQt6-WebEngine` dans votre terminal avant de le lancer.

Python

```python
import sys
import os
from PyQt6.QtWidgets import QApplication, QMainWindow
from PyQt6.QtWebEngineWidgets import QWebEngineView
from PyQt6.QtCore import QUrl

class CartographieApp(QMainWindow):
    def __init__(self):
        super().__init__()

        # Configuration de la fenêtre principale
        self.setWindowTitle("Explorateur Spatial - Sentinel-2")
        self.setGeometry(100, 100, 1280, 720) # Fenêtre en 720p par défaut

        # Création du composant navigateur web
        self.navigateur = QWebEngineView()

        # Récupération du chemin absolu vers le fichier index.html
        chemin_dossier_actuel = os.path.dirname(os.path.abspath(__file__))
        chemin_html = os.path.join(chemin_dossier_actuel, "index.html")

        # Formatage de l'URL pour être lue correctement par QWebEngineView
        url_locale = QUrl.fromLocalFile(chemin_html)

        # Chargement de la page HTML
        self.navigateur.setUrl(url_locale)

        # Définition du navigateur comme widget central
        self.setCentralWidget(self.navigateur)

if __name__ == "__main__":
    # Initialisation de l'application
    app = QApplication(sys.argv)

    # Création et affichage de la fenêtre
    fenetre = CartographieApp()
    fenetre.show()

    # Boucle d'exécution principale
    sys.exit(app.exec())
```

Souhaitez-vous que je vous montre comment compiler ce script Python en un fichier exécutable indépendant (`.exe`) pour pouvoir le partager ou le lancer sans passer par le terminal ?

Gemini est une IA et peut se tromper.